Business Intelligence Systems

INFO 4555 5556

Fall 2025

ETL



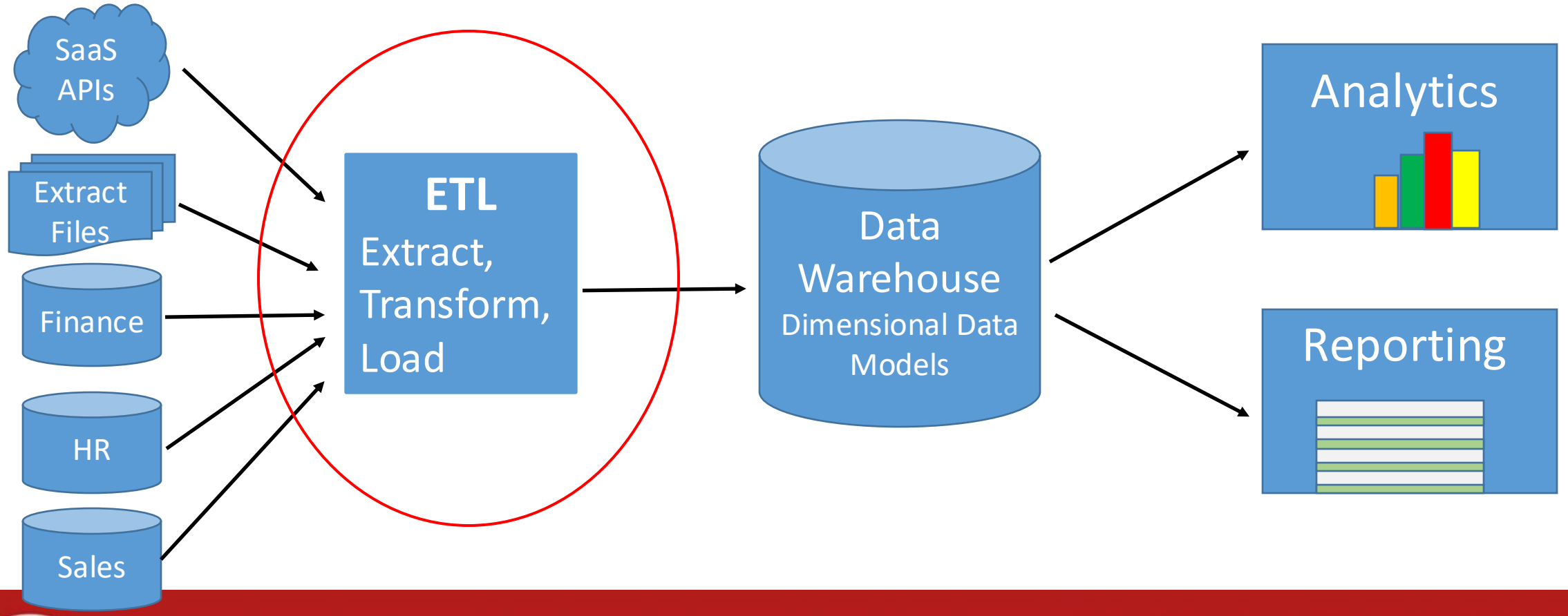
Cornell University

ETL

ETL (Extract Transform Load) is the process of extracting needed data from source data systems, transforming & cleaning data, and finally loading into presentation tables to be used by the customers.

- ETL processing generally involves many steps, sometimes stage tables, in order to get data into its final warehouse ready format
- ETL design involves many decisions, such as where to host the transformation process? Source system, dedicated transformation server, or the target DW server
- Kimball breaks the ETL design process down into a 10 step process (covered in these slides)

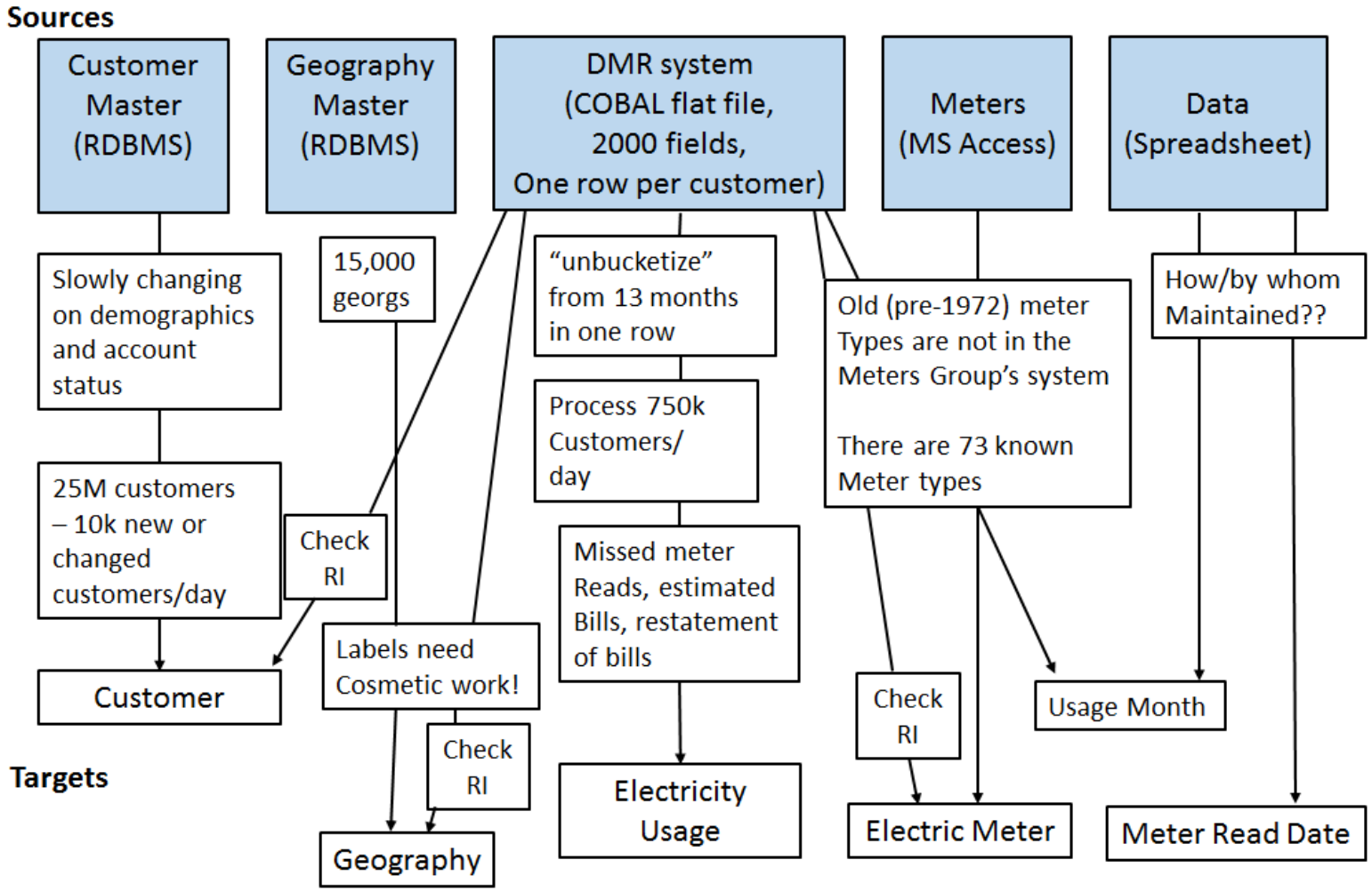
ETL is the heart of DW/BI systems



Step 1: draw the high level plan

- Plan should identify source systems, either physically separate systems or grouping of tables if coming from a large transactional system
- Plan should map to the general subject areas in the DW
- May contain known filters, or cleanup steps that will be needed

Kimball high-level data staging plan schematic



The Data Warehouse Toolkit, Ralph Kimball, Margy Ross – figure 20-1

Step 2: Choosing an ETL Tool

- ETL tools are expensive and have a significant learning curve, however, the benefits to the organization include:
  - Self-documentation that comes from using a graphical tool
  - Metadata foundation for all steps of the ETL process
  - Version Control for multi-developer environments and for backing out and restoring consistent versions
  - Advanced transformation logic, such as fuzzy matching algorithms, integrated access to name and address deduplication routines, and data mining algorithms
  - Improved system performance at a lower level of expertise.
  - Sophisticated processing capabilities, including automatically parallelizing tasks, and automatic fail-over when a processing resource becomes unavailable
  - One-step conversion of virtualized data transformation modules into their physical equivalents

When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

Advantages of using an ETL Tool, or Data Warehouse automation tool, over hand coding ETL, include all of the following, except:

A: Metadata foundation for all steps of the ETL process

B: Improved system performance at a lower level of expertise

C: Self-documentation that comes from using a graphical tool

D: ETL Tools are known to have a very low learning curve

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Step 3: Develop Default Strategies

- Develop a set of default strategies for the common activities in the ETL system, based on the abilities of your ETL tool and organization needs:
  - Extract from each major source system
  - Archive extracted and staged data
  - Police data quality for dimensions and particularly facts
  - Manage changes to dimension attributes
  - Ensure the data warehouse and ETL system meet the system availability requirements
  - Design the data auditing subsystem
  - Organize the ETL staging area

Step 4: Drill down by target table

- Development of detailed plan for the population of each target table
- Source-to-target attribute mappings
- Thorough data profiling
- Understanding of the necessary data transformations for each table and column
  - Data is complete
  - Hierarchies are complete and consistent
  - How are null values handled

Develop the ETL Specification Document

- Should include detailed specifications for each table including the following:
  - Table design (column names, data types, keys, and constraints)
  - Historic data load parameters (number of months) and volumes (row counts)
  - Incremental data volumes, measured as new and updated rows per load cycle
  - Load frequency
  - Handling of slowly changing dimension (SCD) changes for each dimension attribute
  - Table partitioning, such as monthly
  - Overview of data sources, including a discussion of any unusual source characteristics, such as an unusually brief access window.
  - Detailed source-to-target mapping
  - Source data profiling, including at least the minimum and maximum values for each numeric column, count of distinct values in each column, and incidence of NULLs
  - Extract strategy for the source data (flat file, data dump, system APIs, or direct database query)

ETL Specification Document (continued)

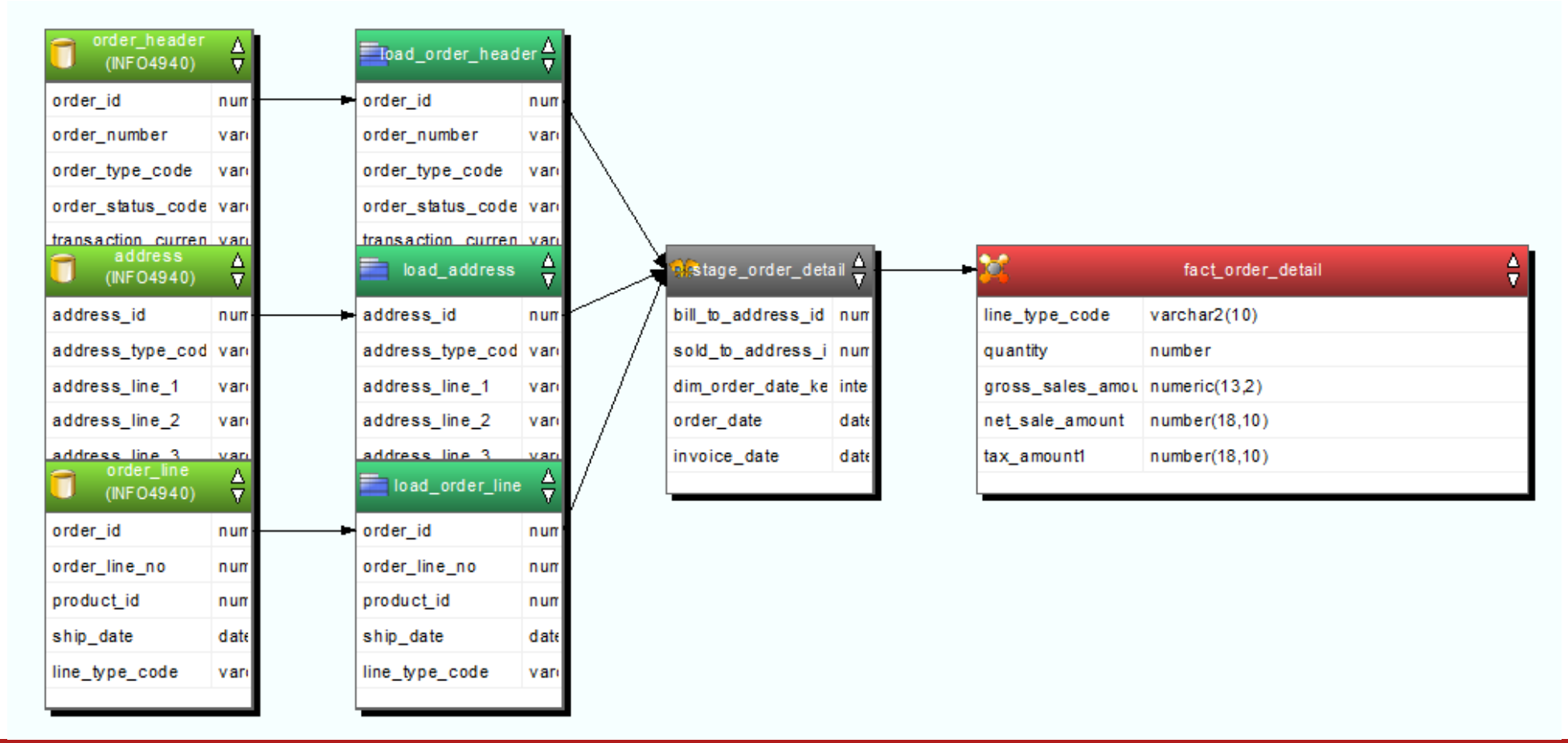
- Dependencies, including which other tables must be loaded before this table is processed.
- Document the transformation logic.
- Preconditions to avoid error conditions.
- Cleanup steps, such as deleting work files.
- An estimate of whether this portion of the ETL system will be easy, medium, or difficult to implement.

Sample ETL Source Target Mapping Doc

Source System/ Schema	Source Table	Source Column	Target Table	Target Column	Transformation Logic	Notes
			DIM_PRODUCT	Product_id	Surrogate key assigned by ETL	Primary key
sales	PRODUCT	Product_name	DIM_PRODUCT	Product_name		
sales	PRODUCT	Product_code	DIM_PRODUCT	Product_code		Natural key from Sales System
sales	PRODUCT_LINE	Prod_line_desc	DIM_PRODUCT	Product_Line_Description		
sales	PRODUCT_GROUP	Prod_group_desc	DIM_PRODUCT	Product_group_name		
sales	ORDER_LINE	Net_sale_amount, Tax_amount	FACT_ORDER_DETAIL	Gross_sales_amount	Net_sale_amount + tax amount	Derived field
...	...	...	...	...	...	...



Sample Source Target Mapping Diagram



When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

How are you feeling about the ETL Specification Document?

Good – All makes sense

OK – I'll figure it out

Bad – Not understanding all of the details

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Develop a Sandbox Source System

- During ETL development it is preferred to have a static copy of the source system data
- Removes risk to production source system of bad ETL queries
- Easier to validate ETL results if source system is not changing

When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

All of the following are reasons to use a sandbox "static" source system for ETL development, except:

A: Removes risk to production source system of bad ETL queries

B: Easier to validate ETL results because source system is not changing

C: Great for testing Type 2 SCD code

D: Allows consistent source data during the development process

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Develop One-Time Historic Load Processing

- Start by developing a process for loading historical data (initial load)
- Some portions of the load process may use initial load process going forward (destructive load)
- Often times the ongoing ETL process varies from the initial load process because it is designed to be more incremental in its processing

Step 5: Populate Dimension Tables with Historic Data

- Populate type 1 dimension tables
- Simple data transformations (such as default values for NULLs)
- Combine from separate sources (de-duplicate values, fix misspellings)
- Decode production codes (replace, or add text equivalents)
- Validate many-to-one and One-to-One Relationships

```
SELECT Product_SKU, count(*) as Row_Count, count (distinct Product_Model)
as Model_Count
FROM StagingDatabase.Product
GROUP BY Product_SKU
HAVING Count(distinct Product_Model) > 1;
```

Dimension Table initial load

- Following recommendations for bulk loading data
  - Turn off logging
  - Bulk load in fast mode
  - Presort the file (speeds index builds)
  - Transform with caution (often times faster to transform data after the bulk load, ELT)
  - Truncate table before full refresh (much faster than a delete)

Dimension Table initial load (continued)

- Load Type 2 Dimension Table History
  - If attribute history is maintained in the operational system, it may be possible to create the SCD history for the initial load of the dimension table
  - The logic to populate the SCD records from the operational system can be complex, will need an ETL tool, or procedural language (more than SQL)

Step 6: Perform the Fact Table Historic Load

- Fact table loads are generally incremental (only loading new fact transaction or snapshot records) or partial (single month or year refreshed)
- The initial “historic” load of a fact table can take a lot of time

Null Fact Values

- All foreign keys in a fact table should contain valid keys to dimension records (no nulls)
- If there is NOT a valid match to a dimension record, there should still be a valid foreign key (generally 0) which joins to a dimension record of “N/A”
- For the actual facts/measures, nulls are allowed because they are handled appropriately by calculations
- Sometimes operational systems will assign a value such as “-1” to null values, these should be replaced will NULLS in the fact table

When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

How are you feeling about Null Fact Values?

Good – no problem

OK – good enough

Bad – would like to discuss further

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Pipeline the Dimension Surrogate key Lookup

- Ensure Referential Integrity (RI) is maintained between the fact table and the dimension tables (all facts have associated dimension records)
- The surrogate key pipeline is the final step before loading the records into the fact table
- Dimension table processing must be complete
- If there are type 2 dimensions, the surrogate key lookups, must include BETWEEN date logic to get the appropriate SCD surrogate key from the dimension
- Often times the process used to determine the appropriate dimension surrogate keys is to join the fact stage table to the appropriate dimensions by the business key.

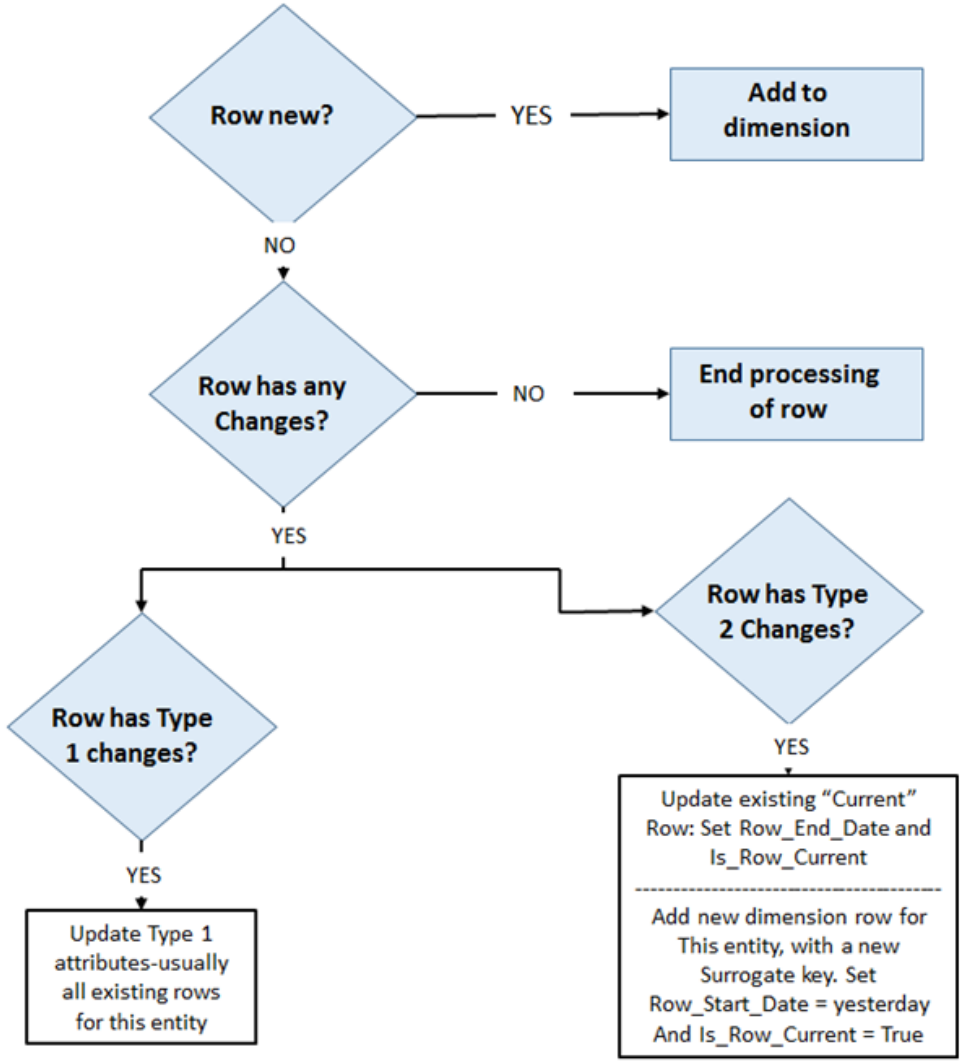
Fact Table Loading – to improve load performance:

1) Disable Foreign key (referential integrity) constraints between the fact table and each dimension table before loading data
2) Drop or disable indexes on the fact table
3) Load the data using fast-loading techniques (direct path, bulk loader)
4) Create or enable fact table indexes
5) Enable foreign key constraints

Step7: Dimension Table Incremental Processing

- Identifying New and Changed Dimension Rows
  - If possible, extract files sent to the DW for processing should only contain change records
  - Ideally operational system contains tags to identify when a record was last updated. (Must be reliably maintained by operational system)
  - Often times the only method available for determining record changes is to compare all columns for each record
  - If a full comparison of all columns and all rows is too expensive (takes too long) may use a hash value compare
- Process Changes to Dimension Attributes (according to SCD types selected)

Logic flow for handling dimension updates



The Data Warehouse Toolkit, Ralph Kimball, Margy Ross – figure 20-5

When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

How are you feeling about Dimension Table Incremental Processing?

Good – no problem

OK – good enough

Bad – would like to discuss further

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Step 8: Fact Table Incremental Processing

- Most data warehouse databases are too large to entirely replace the fact tables in a single load window
- New and updated fact rows are incrementally processed
- Incremental processing must be fully automated, unlike the initial historic load

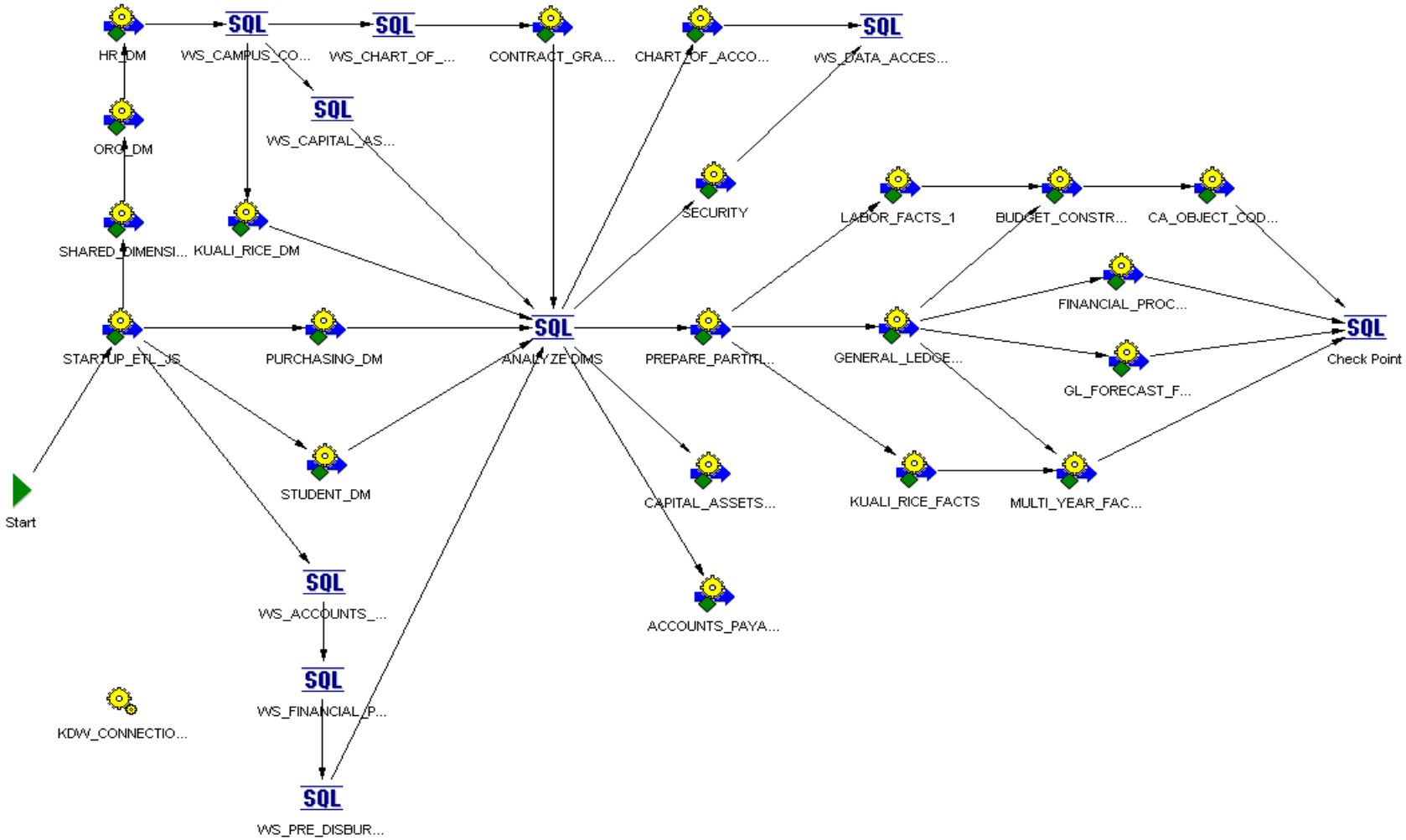
Options for handling referential integrity violations:

- Halt the load
- Throw away error rows
- Write error rows to a file or table for later analysis
- Fix error rows by creating a dummy dimension row and returning its surrogate key to the pipeline
- Fix error rows by mapping to a single unknown member in each dimension (not recommended)

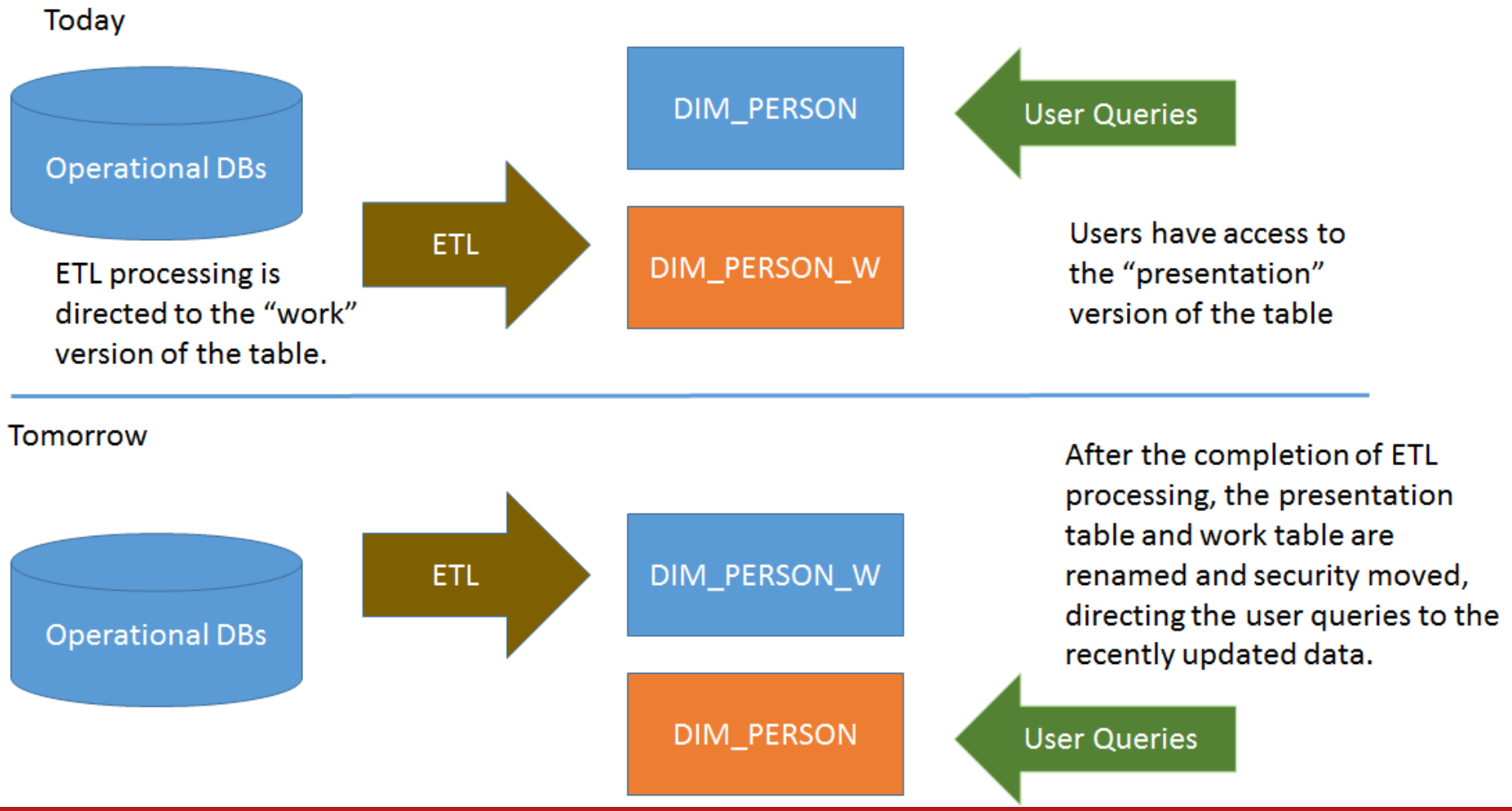
Parallel processing

- The ETL can be broken into multiple steps and dependencies defined.
- The Database can handle parallel execution for some processing such as index builds.
- Parallel structures (multiple copies of the database, tables, or partitions may be used to improve DW availability during load processing)

ETL Process Parallelization – Dependency Mapping



Parallel copies of tables for high data availability



Step 9: Aggregate Table Loads

- Aggregate tables often included in DW for performance reasons
- After updates to the underlying atomic level facts occur the aggregate tables must be updated accordingly to avoid mismatched results

Step 10: ETL System Operation and Automation

- ETL processing should run in a “lights out” mode, not requiring human intervention
- Scheduling of ETL processing should handle dependency checking
- Automatically Handle Predictable Exceptions and Errors
- Gracefully Handle Unpredictable Errors

When poll is active, respond at **pollev.com/info4555**

Text **INFO4555** to **22333** once to join

How are your feeling about the topic of ETL?

Good – I'm ready to tackle the "hands on" ETL lab next week.

OK – I'm sure it will make more sense after the "hands on" ETL lab

Bad – ETL is worse than dimensional modeling

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**